

Summer Undergraduate Research Fellowship (SURF) Final Report

Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly

Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco

Department of Computer Science, University of Nevada, Las Vegas

Faculty Mentor: James Andro-Vasko, Ph.D., Assistant Professor in Residence and

Undergraduate Coordinator, Department of Computer Science

Office of Undergraduate Research, UNLV — Fall 2026

1. Project Description

1.1 Main Objectives of the Research

This project did not begin as research. It began as a fix. Students in UNLV's introductory C++ course (CS 135) and x86-64 assembly course (CS 218) lose the first week of the semester getting a compiler, an assembler, a debugger, and an editor working on dozens of different laptops, and students with Apple Silicon Macs often cannot run the course's virtual machine at all. Before the fellowship I built two Docker images to remove that week. Each packs a browser-served Visual Studio Code editor (code-server) with the exact toolchain the course expects, so a student installs only Docker Desktop, runs one command, and opens a browser tab. Everything runs on the student's own machine. The approach was chosen for its simplicity, and it worked on the machine it was built on.

The fellowship turned that artifact into research by asking two questions it could not answer on its own. Is the promise actually true on the computers students own, not only on the

developer's laptop? And what does the design give up compared with how other institutions have solved the same problem? The objectives followed from those questions:

1. Measure, with one published test script run identically everywhere, whether the images behave the same across Windows, macOS, and Linux on both Intel/AMD and ARM processors: startup time, download size, memory, compile time, debugger availability, and persistence of student files.
2. Preserve the course's real x86-64 instruction set for assembly work while still supporting ARM hosts, and document precisely where that support ends.
3. Position the design within the research literature and within UNLV's own history of student infrastructure, and produce a protocol for a future classroom study.

1.2 Summary of the Problem and Why It Is Important

The problem is old, well documented, and expensive. Harvie et al. measured 75 to 100 minutes of class time per semester lost to software installation before their department standardized environments [6]. Valstar et al. found that 90% of their students owned Windows or Mac laptops while the autograder ran Linux, so much of the support load was environment mismatch rather than course content [3]. Fernalld et al. reported that students with M1 and M2 MacBooks could not run the x86 virtual machines their courses assumed [4].

UNLV has followed the same path as the institutions in that literature, one step behind. For two decades students reached course tools through shared departmental Linux servers over SSH, commonly with PuTTY, and a submit script that required logging in even from lab machines. Those servers (bobby, sally, cardiac) are decommissioned, the wiki that documented them was deleted in a 2025 site rebuild, and current guidance points to a VPN-gated host and to virtual desktops whose local state is "PERMANENTLY DELETED" unless saved to a network

drive [12]. CS 218 today distributes a VirtualBox image for Intel machines and an experimental Fall-2023 image for Apple Silicon that emulates a whole x86-64 computer, slowly enough that the course warns VS Code may be unusable; the official advice for M1/M2 owners is to use the TBE-A311 lab with a USB drive. Every approach hands the setup burden to someone: the department, the student's hardware, a cloud vendor, or the student's privacy. The question is whether a local container can leave the student carrying almost nothing after one Docker installation, and whether that claim survives contact with real hardware. Several hundred CS 135 and CS 218 students a year are affected, and setup friction falls hardest on students with the cheapest or newest machines.

1.3 Specific Research Questions

Because the images had not yet been used by a full course section, this cycle addressed the questions answerable before deployment:

- **RQ1 (Reproducibility).** Do the published images run the documented workflows, including real CS 218 assignments?
- **RQ2 (Portability).** Which host operating systems and processors can run the assembly environment, and what limits remain?
- **RQ3 (Resource cost).** What startup, storage, memory, and compile-time costs does it impose, and how large is the emulation overhead on ARM?
- **RQ4 (Persistence).** Do student files survive replacing the container or image?
- **RQ5 (Architecture fidelity).** How far can an amd64-only assembly environment, including its debugger, reproduce course behavior on ARM hosts?

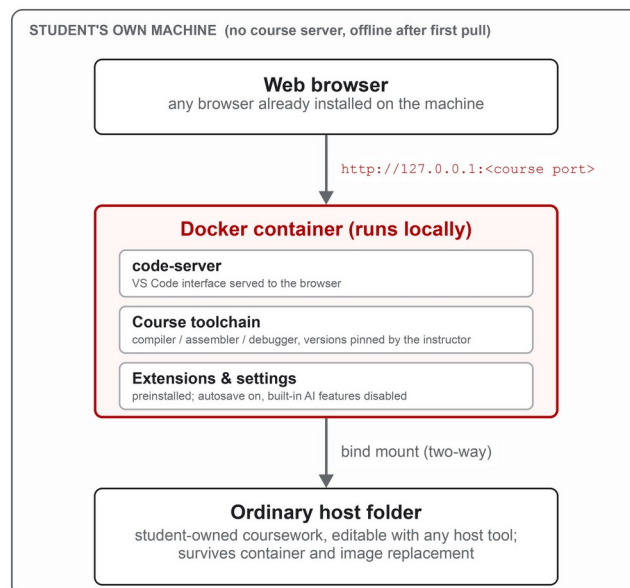
Questions about setup success rates, support burden, and student experience require a classroom and are deferred to the study in Section 4.2.

2. Methodology

2.1 Design and Protocol Description

System. Both images are built from Ubuntu 22.04 and serve code-server 4.126.0 on a localhost-only port numbered after the course (8135, 8218). The C++ image ships g++ 11.4 and runs natively on Intel/AMD and ARM. The x86 image ships yasm, nasm, ld, and gdb 12.1 and is deliberately amd64-only, because the course's programs can only assemble and run as x86-64 Linux binaries; on ARM hosts Docker runs it through a translation layer. Student work lives in an ordinary host folder (a bind mount), never inside the container, so a container can be deleted or updated without touching coursework. Starter files are seeded only into an empty folder, no automatic restart policy is set, and every AI feature in the editor is disabled per course policy (Figure 1). Dockerfiles, handouts, and test scripts are public [11].

Figure 1. One course IDE. Everything runs on the student's machine; the only network dependency is the initial download.



Evaluation. The same published script (`ci-test.sh`) is run, unmodified, on four host configurations, and every reported number is transcribed from the JSON file it writes. The script performs a student’s first day and then some: a timed download recording the image’s digest; a timed start until the editor answers; a check that starter files appear once; three timed assemble-link-run cycles; a scripted gdb probe recorded as working or broken; idle memory; a timed build and run of four real CS 218 assignments with peak memory sampled; and destruction and recreation of the container to verify that student files survive. Because the same image digest is downloaded everywhere, any difference between hosts is due to the host, not the image. Three hosts were rented from public clouds so that a fresh, blank machine of each kind could play the role of a student laptop; the fourth was an owned MacBook. Windows Server on AWS turned out to be outside Docker Desktop’s support, so the Windows cell moved to Azure, the only cloud that rents real Windows 11.

2.2 Materials, Sample Size, and Sampling Description

The unit of analysis is a host configuration; no human participants were involved. The image under test was `seancnc/unlv-x86-ide` at one pinned digest, 552 MB, verified identical on every host. The C++ image runs natively everywhere and was covered by the project’s separate release checks. Table 1 lists the four hosts. The workload was four actual CS 218 assignments spanning the course: two pure-assembly programs (`ast3`, `ast04`), a C++ driver calling assembly procedures with file I/O (`ast06`), and a multithreaded program with an assembly kernel computing a checkable answer (`ast12`), the sharpest test of whether emulation runs code correctly. The MacBook was deliberately the base 16 GB M1 Pro rather than a 64 GB development machine, because that is what a budget-conscious student buys. The C++ image

was also used once in a three-student class for high school students; no metrics were collected, and it is reported only as a first deployment.

Table 1. The four hosts. Cells 1 and 2 are same-size Intel/ARM siblings, so their comparison isolates emulation.

Cell	Host	OS / Docker	x86 image mode
1 Linux amd64	AWS m8i.large, Intel Xeon, 2 vCPU / 8 GB	Ubuntu 24.04, Docker Engine 29.1	native
2 Linux arm64	AWS m8g.large, Graviton (ARM), 2 vCPU / 8 GB	Ubuntu 24.04, Docker Engine 29.1, QEMU	emulated
3 Windows amd64	Azure D4s_v5, Intel Xeon, 4 vCPU / 16 GB	Windows 11 Pro 24H2, Docker Desktop, WSL2	native
4 macOS arm64	MacBook Pro 14-inch (2021), M1 Pro, 16 GB, owned	macOS 26.4, Docker Desktop 29.2	emulated

Nine peer-reviewed papers were read in full and summarized in a shared annotated file: the direct lineage of containerized course environments [1, 3, 4], evidence on assembly courses and virtual machines [5, 6], the history of local course appliances [2, 7], and the contrast cases of classroom monitoring and editor-embedded AI [8, 9]. A separate investigation using live web fetches, DNS and SSH probes, an archived 2012 lab manual, and CS 218 course pages reconstructed UNLV’s student-infrastructure history, with every claim labeled by evidence strength.

2.3 Analysis Framework and Technique

The analysis is descriptive and comparative. Each host yields pass/fail outcomes (file seeding, persistence, each assignment, debugger) and timings in seconds and memory in mebibytes. Native hosts must pass every check; on emulated hosts a broken debugger or failed assignment is recorded as a finding, not a failure, because documenting that boundary is the purpose of those cells. Emulation overhead is the emulated time divided by the native time on the identical workload, using cell 1 as the baseline because cells 1 and 2 differ only in processor.

Timer resolution is 0.1 s, so ratios are reported as order-of-magnitude figures. Every anomaly was handled by one rule: rerun to reproduce, capture diagnostics, archive everything verbatim, and report it. Claims were sorted into those supported by the repository, those requiring the measurements in this report, and those requiring classroom data; this report makes no claim from the third group.

2.4 Specific Undergraduate Researcher (URM) Contributions to the Work

I designed and built both images, their settings, starter files, and the student handouts, and published them to Docker Hub through an automated build pipeline. I wrote the measurement script and chose and verified the four assignments. I wrote the AWS and Azure runbooks, provisioned every cloud machine by hand, diagnosed the failed launches, and personally ran the record runs, the ARM crash reproduction, the follow-up experiment, and the Windows 11 run. I read and annotated the nine papers, wrote the literature synthesis, carried out the institutional-history investigation, and drafted the experiment plan, results archive, this report, and the symposium poster. Dr. Andro-Vasko provided the course context and requirements for CS 135 and CS 218, guidance on which claims the evidence could support, and feedback on the research framing.

3. Major Findings

3.1 Results and Key Findings

The same image behaved identically on three operating systems and two processor families, with one documented exception. Table 2 summarizes the runs. The same 552 MB image downloaded on every host. Starter seeding and file persistence passed everywhere (RQ4). All four CS 218 assignments built and ran correctly on all four hosts, including the multithreaded program producing its checkable answer under emulation (RQ1). The only behavioral difference

is the debugger: gdb worked on both native hosts and was broken on both emulated hosts, through two independent translation layers (RQ5). Under translation a gdb script still appears to succeed and writes its output file, but with labels and no register values, which is why the handouts warn Apple Silicon owners not to trust an apparently successful debugger run.

Table 2. Selected results for the x86 image (all values from the unmodified published script).

Metric	Cell 1 Linux (native)	Cell 2 Linux ARM (emulated)	Cell 3 Windows 11 (native)	Cell 4 M1 Mac (emulated)
Debugger (gdb) probe	working	broken	working	broken
Image size (MB)	552	552	552	552
Start until editor ready (s)	0.5	6.7	0.6	4.8
Idle memory (MiB)	54	255	56	263
Peak memory during coursework (MiB)	not captured	334	90	358
ast06 build (s)	0.2	2.8	0.5	2.5
ast12 build (s)	0.3	3.6	0.6	3.1
All four assignments pass	yes	yes	yes	yes
Files survive container replacement	yes	yes	yes	yes
Checks passed / failed	13 / 0	12 / 0	13 / 0	12 / 0

The cost is small on native hardware and modest under emulation (RQ3). On Intel/AMD hosts the editor is ready in about half a second and idles at roughly 55 MiB; the heaviest assignment builds in under a second. Under emulation startup rises to five to seven seconds and peak memory to about 350 MiB, still a small slice of an 8 GB laptop. Emulation cost roughly one order of magnitude on compile-heavy steps (about 14x and 12x on the two largest builds under QEMU, about 12x and 10x under Docker Desktop on the Mac), yet every emulated build finished in under four seconds. Download time was dominated by network, not platform, and happens once.

The support boundary is a translator boundary, not an operating-system boundary (RQ2). The image runs wherever the layer hosting Docker’s Linux environment either needs no

translation or receives a high-quality x86 translator. Windows and Linux on Intel/AMD are native and get everything; Windows 11 was in fact the strongest host in the matrix, with sub-second starts and a working debugger, which matters because Windows dominates student laptops. Apple Silicon works because Apple supplies its Rosetta translator to Docker’s Linux virtual machine. On ARM Linux, Ubuntu 24.04’s stock QEMU translator crashed while translating the editor itself, reproduced with diagnostics archived; the identical machine passed every check after switching to Docker’s own translator build, so the crash is tied to a translator version, not to the image, and the one-line fix is in the student documentation. Windows on ARM could not be rented in any cloud (rented virtual machines cannot nest the virtualization it requires) and Intel Macs were unavailable; both are stated as expectations, not results.

4. Broader Implications

4.1 Interpretation of the Findings in Relation to Related Studies

Where this sits. Harvard’s CS50 is the best-documented history of a course environment: a shared cluster, then cloud virtual machines, then a client-side appliance VM on students’ laptops, then Docker containers behind a cloud editor, and today VS Code on GitHub Codespaces used by more than 700,000 people [1, 2]. Malan’s account of the appliance era reads like a list of the problems containers were built to solve: a 2 GB image, hours to rebuild, a fifth of students finding it slow, disk corruption when a laptop lid closed [2]. Codespaces fixed those problems by moving the container to GitHub’s servers, at the price of an account, a network connection, and students who finished the course “not really sure how to set up an IDE on my computer directly” [1]. This project takes the same container and browser editor and puts the container back on the student’s machine. The measurements are the evidence that the trade is

affordable: seconds instead of minutes to start, 552 MB instead of a 19 GB virtual disk, and permanent student ownership of the environment. Table 3 places the alternatives side by side.

Table 3. Who carries the setup burden under each approach.

Approach	Who carries the burden	What it costs the student
Shared login servers over SSH (UNLV bobby/sally/cardiac)	The department	Network and VPN dependence; the environment and its documentation vanish when servers are retired
Course virtual machines (CS 218 images; CS50 Appliance [2]; Reading’s QEMU lab [5])	The student’s hardware	Multi-gigabyte images, fixed RAM reservations, slow boots, disk corruption; unusable or very slow on Apple Silicon
Cloud-hosted browser IDE (CS50 on Codespaces [1])	A vendor, a grant, an account	Internet required; environment disappears with the account; no path to a local setup
Monitored classroom platform (CodeDive, South Korea [8])	The institution’s cluster and the student’s privacy	Every process and one-second typing pause recorded; consent that cannot realistically be withheld
Local container with browser editor (this project; cf. [3, 4])	The student, once: install Docker Desktop	One download and one command; offline afterward; files stay on the laptop; debugger unavailable on Apple Silicon

The two contrast cases. CodeDive, from Jeonbuk National University and partners, is the modern centralized counterpart: browser VS Code in per-student containers on a cluster, with a kernel-level tracer and a code snapshot after every one-second typing pause, 24,845 snapshots from 95 students in two weeks, used for dashboards and plagiarism screening [8]. That is a rational answer to assessment in the era of large language models, and it gives instructors visibility this project deliberately forgoes; its authors also note it needs a data-protection framework they have not yet built. This design takes the opposite corner on purpose: nothing leaves the laptop until the student submits, enforced by architecture rather than policy. Likewise, CS50’s guardrailed AI tutor hallucinated with confident tone and reached 88% accuracy [9]; these images disable every editor AI feature so that AI use is governed by course policy, a considered position rather than an omission.

Virtual machines and the debugger trade-off. Cadenas et al. replaced hardware boards with a QEMU emulator and raised lab marks significantly over six cohorts, but they changed the course's instruction set to whatever the emulator provided in exchange for a working debugger [5]. UNLV's own Apple Silicon image is the same corner: it emulates a whole x86-64 computer, keeps the debugger, and is too slow for VS Code. This project takes the opposite corner: it keeps the real x86-64 instruction set, runs roughly ten times slower than native on builds but still in seconds, and loses the debugger on ARM. Native hardware gets both; nothing on ARM yet does, and this evaluation makes that trade-off measured and explicit. Choosing a container architecture to preserve a course's real instruction set, and characterizing debugger behavior under user-mode emulation, has no direct precedent in the nine papers reviewed. Finally, the two closest local-container precedents, UCSD [3] and Florida Tech [4], found the same honest limitation this design inherits: installing Docker itself is the hardest step, and it falls hardest on first-year students.

4.2 Potential Future Directions for Research

- **A classroom study.** Deploy the images in CS 218 and CS 135 with IRB approval and measure the funnel from enrollment to attempted setup, successful setup, continued use, and survey response, using installation-minutes, support-traffic, and setup-anxiety instruments adapted from the literature [3, 4, 6]. Grade comparisons must be read as association, not cause.
- **A hybrid assembly image for ARM.** The ARM crash occurred while translating the editor, not the student's program. A variant that runs the editor natively and translates only student binaries could restore debugging through QEMU's built-in gdb stub, covering Apple Silicon and Windows-on-ARM at once.

- **Generalizing the method.** The one-script, rent-four-computers evaluation is course-agnostic and could vet environments for CS 370 (Operating Systems) and other departments; physical Intel Mac and Windows-on-ARM laptops would close the two untested cells.

5. Reflection

5.1 Impact on Personal and Professional Development

This fellowship changed how I think about the difference between building something and knowing something about it. I came in with working software that I believed ran everywhere. The research process replaced that belief with evidence, and the most important lesson was that a promise tested only where it succeeds is an advertisement, not a result. Designing the experiment taught me to isolate the one variable I cared about, to choose baselines so nothing else changed, to treat a negative result as a finding rather than an embarrassment, and to archive every raw output so someone else could check my numbers.

Reading the literature in a prescribed order, from Harvard's history through the virtual-machine studies to the Korean monitoring platform, gave me a vocabulary for what I had built and showed me that UNLV had followed the same path as everyone else but stopped one step early. That reframed the project from a tool I made into a position in a design space with trade-offs I could name: local versus cloud, telemetry versus privacy, instruction-set fidelity versus debugger availability. Professionally, provisioning every cloud machine by hand before automating anything gave me a working command of AWS and Azure I did not have in May, and writing runbooks and weekly check-ins that my mentor could act on made me a clearer technical writer and a better collaborator.

This applied research showed me how people are facing the same problems across different parts of the world. Sharing knowledge and findings is important so that no effort is duplicated and that technology is always on the frontier. It helped me stop and think about how the things that I am researching and building fit into the bigger picture and how people do things. It puts into perspective how and why people do things that they do and have respect for tried and true methods while also keeping in mind progress and change.

It opened my eyes to how much impact and lives are affected. Doing this research, I had the chance to share the tools with a high-school section for the course. It was amazing to see how not one student had a question regarding environment setup and everyone was able to focus on learning the material. We start small today with research, but there is potential to scale and change how education is structured and standardized for everyone to focus on the important things.

5.2 Future Plans for Research Involvement

My immediate plan is to carry this project into its second stage with Dr. Andro-Vasko: deploy the images in CS 218 and CS 135, obtain IRB approval for the instruments prepared this summer, and collect the classroom data these pre-deployment results were designed to make interpretable. I intend to present the poster at the undergraduate research symposium and to prepare the system paper and study protocol for a computing-education venue such as SIGCSE or ITiCSE, where the papers that shaped this work were published. Longer term, this fellowship confirmed that I want applied systems research, solving a concrete problem for a real population and measuring it honestly, to be part of my career, and I plan to pursue graduate study in computer science with that focus. There are a lot of problems put in the backlog that can now be

addressed with recent advancements in technology and AI. Doing applied research and directly having an impact and causing improvements in people's lives excite me.

References

- [1] David J. Malan. 2024. Containerizing CS50: Standardizing Students' Programming Environments. In Proceedings of the 2024 Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024). ACM, Milan, Italy.
<https://doi.org/10.1145/3649217.3653567>
- [2] David J. Malan. 2013. From Cluster to Cloud to Appliance. In Proceedings of the 18th ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '13). ACM, Canterbury, UK.
- [3] Sander Valstar, William G. Griswold, and Leo Porter. 2020. Using DevContainers to Standardize Student Development Environments: An Experience Report. In Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '20). ACM, Trondheim, Norway.
<https://doi.org/10.1145/3341525.3387424>
- [4] Kourtnee Fernalld, TJ OConnor, Sneha Sudhakaran, and Nasheen Nur. 2023. Lightweight Symphony: Towards Reducing Computer Science Student Anxiety with Standardized Docker Environments. In Proceedings of the 24th Annual Conference on Information Technology Education (SIGITE '23). ACM, Marietta, GA.
<https://doi.org/10.1145/3585059.3611432>
- [5] J. O. Cadenas, R. S. Sherratt, D. Howlett, C. G. Guy, and K. Lundqvist. 2015. Virtualization for Cost-Effective Teaching of Assembly Language Programming. IEEE Transactions on Education 58, 4 (2015), 282–288. <https://doi.org/10.1109/TE.2015.2405895>

- [6] David P. Harvie, Jason R. Cody, Christopher Morrell, and Tanya T. Estes. 2019. Using Virtual Machines to Enhance the Educational Experience in an Introductory Computing Course. In Proceedings of the 20th Annual Conference on Information Technology Education (SIGITE '19). ACM, Tacoma, WA, 28–32.
<https://doi.org/10.1145/3349266.3351401>
- [7] Oren Laadan, Jason Nieh, and Nicolas Viennot. 2010. Teaching Operating Systems Using Virtual Appliances and Distributed Version Control. In Proceedings of the 41st ACM Technical Symposium on Computer Science Education (SIGCSE '10). ACM, Milwaukee, WI, 480–484.
- [8] H. Park, Y. Kim, K. Lee, S. Jin, J. Kim, Y. Heo, G. Kim, and E. Kim. 2025. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring for Programming Education. Applied Sciences 15, 19 (2025), 10403. <https://doi.org/10.3390/app151910403>
- [9] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. 2024. Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024). ACM, Portland, OR.
<https://doi.org/10.1145/3626252.3630938>
- [10] Ed Jorgensen. 2024. x86-64 Assembly Language Programming with Ubuntu (Version 1.1.58). University of Nevada, Las Vegas. <http://www.egr.unlv.edu/~ed/assembly64.pdf>
- [11] Sean Cuenco. 2026. UNLV Docker IDEs: source repository, handouts, measurement script, and record-run evidence. <https://github.com/seancnc003/UNLVDockerIDEs>. Images: hub.docker.com/r/seancnc/unlv-cpp-ide and hub.docker.com/r/seancnc/unlv-x86-ide.

- [12] UNLV Department of Computer Science. 2025. Student Center: Remote Access and File Storage. <https://tux.cs.unlv.edu/remote-access.html> (accessed August 16, 2026).